

Code-Layout, Readability and Reusability

Date	1 st july 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard PEP 8 styling guidelines are applied with strict 4-space indentation across all Python scripts and clean indentation inside the HTML/Jinja2 template blocks.
2	Proper File Structure	Files and folders are logically organize	Yes	Clear separation maintained: data scripts live in a processing directory, localized templates live within a dedicated /templates folder, and the model weights reside as an asset (model.pkl).
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Cryptic labels are avoided. Multi-table data sets use clear titles like merged_credit_df and preprocessing indicators are clearly labeled like label_encoder_occupation.
4	Function / Method Names	Functions are descriptively named	Yes	Blocks use declarative naming mappings like load_and_clean_data(), handle_class_imbalance(), and predict_eligibility().
5	Code Comments	Inline and block comments explain logic.	Yes	Every computational step contains explicit comments detailing the specific data transformations, model parameters (class_weight='balanced'), and Flask routing behaviors.

6	Modular Design	Code is split into reusable functions/m	Yes	The pipeline logic is strictly isolated from the server execution interface, wrapping core preprocessing operations inside self-contained callable functions.
7	No Redundant Code	Duplicate or unused code is removed	Yes	All testing artifacts and legacy looping mechanisms generated during exploratory modeling phases have been stripped out.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	The Flask processing router wraps payload handling inside custom try-except blocks to flag missing parameters or bad inputs without crashing the server thread.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Cleaning & Imputation Engine	Python / Pandas	Executed during initial batch pipeline compilation and reused directly during inference formatting to handle missing strings in real-time.	High

2	Categorical Label Encoding Pipeline	Python / Scikit-Learn	Extracted from training configurations and re-applied inside the dynamic web payload routing structure to parse fresh text arrays.	High
3	Flask Error Handler & Sanitizer Wrapper	Python / Flask	Embedded inside the endpoint framework to handle bad data inputs or parsing failures across different form input routes.	Medium
4	Bootstrap 5 Card Layout Container	HTML5 / CSS3	Written once as a base responsive template structure and shared across both the collection form and the final verification text display template.	High
5	Model Inference Connector Interface	Python / Pickle	Serves as the localized weight retrieval module that opens the binary asset (model.pkl) to run classifications inside any fresh scripting sandbox.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Impeccable standard directory layout splitting static assets, HTML templates, and pipeline modules cleanly.
Readability	5	Follows structural formatting rules, self-documenting function naming strategies, and highly descriptive variable labels

Reusability	4	Preprocessing scripts and encoding states are fully modularized, though model loading depends on a static local file name path string.
Documentation / Comments	5	Heavy usage of inline docstrings detailing target binarization criteria, row scaling steps, and backend API routing actions.
Overall Score	4.8 / 5	An exceptionally clean, production-ready development workspace built for seamless group handover and effortless maintainability